# Food Bank Resource Allocation Optimizer

## Reinforcement Learning Project Report

Course Code: CT-469 Reinforcement Learning

Instructor: Dr. Murk Marvi

Department of Computer Science and Artificial Intelligence

NED University of Engineering and Technology

Group Members: AI 22041 Talha Noor, AI 22047 Sufiyan Nadeem, AI 22044 Ali Rana

## Individual Contributions

This section is placed near the start of the report as required by the deliverable instructions.

| Student | Main Contribution | Relevant Files / Components |
|---|---|---|
| AI 22041 Talha Noor | Baseline planning, evaluation comparison support, documentation review, and demo validation. | agents/baselines.py, test.py, report and presentation review |
| AI 22047 Sufiyan Nadeem | RL formulation, environment design review, training/testing, reward tuning, dashboard testing, and final result interpretation. | environment/foodbank_env.py, agents/dqn_agent.py, train.py, test.py, app.py |
| AI 22044 Ali Rana | Web interface checking, result screenshots, experiment verification, and presentation/live demo preparation. | app.py, results/*.png, Streamlit demo, presentation assets |

## Abstract

This project presents a reinforcement learning based decision-support prototype for food-bank resource allocation. The system simulates donated food inventory, recipient families, expiry pressure, nutritional value, dietary restrictions, fairness and budget. A DQN agent learns an allocation strategy and is evaluated against five non-RL baselines: First-Come-First-Served, Greedy Expiry, Nutrition-First, Fairness-First and Random allocation. The final evaluation shows that the DQN agent achieved the highest average positive reward, demonstrating the strongest overall balance between nutrition delivery, fairness, waste control and dietary compatibility.

## 1. Problem Statement

Food banks distribute limited donations under time pressure. Perishable items can expire unused, high-need families may be missed, and simple manual rules may not handle nutrition, dietary constraints and fairness simultaneously. The project goal is to create a self-contained, runnable RL prototype that allocates food more intelligently while still providing non-RL baselines and a browser-based live demo for evaluators.

## 2. System Design

The prototype is organized as a deployable Python repository. It contains a custom Gymnasium-style environment, a DQN-based RL agent, rule-based baselines, standalone training and testing scripts, saved model weights, generated result graphs and a Streamlit web dashboard.

| Component | Description |
|---|---|
| Environment / Simulation | Models food items, quantity, expiry, nutrition, category, recipient family size, dietary restriction, need score, coverage, fairness and mismatches. |
| DQN Agent | Uses a dueling Double-DQN style network to select among allocation strategies. |
| Non-RL Baselines | Fixed rule methods used for comparison under the same environment and random seeds. |
| Web Dashboard | Streamlit app that runs a live allocation comparison and displays inventory/recipient snapshots. |
| Saved Model | models/dqn_foodbank.pt is included so evaluators can run test.py and app.py without retraining. |

## 3. RL Formulation

### 3.1 State Space

The state includes normalized features describing the current food item, inventory pressure, recipient need, dietary category, coverage gap, fairness gap and budget. These variables give the agent enough context to decide whether to prioritize nutrition, fairness, expiry rescue or skipping.

### 3.2 Action Space

Instead of directly selecting a raw family ID from many families, the final prototype uses learnable allocation strategies. The DQN chooses among balanced allocation, nutrition-first allocation, expiry-first allocation, fairness-first allocation, queue-based allocation, or skip. This makes the action space easier to learn and closer to a realistic decision-support tool where the agent recommends a policy style for the current situation.

### 3.3 Positive Reward Function

The reward function was redesigned as a positive scoring system so it is easy for evaluators to interpret. Good allocations earn points for nutrition delivered, improving recipient coverage, reducing unfairness, serving neglected families, matching dietary restrictions and rescuing near-expiry items. Penalties reduce the score for dietary mismatches, unnecessary skips, waste and repeatedly over-serving already-covered recipients. The objective is therefore to maximize a positive multi-objective score, not just a single metric.

## 4. Algorithm and Training

The RL model is based on DQN with improvements that make learning more stable. The network uses a dueling architecture, separating state value from action advantage, and Double-DQN target calculation to reduce overestimation. The model is trained using experience replay and a target network. The final training curve shows a clear upward trend in the 20-episode moving average, indicating that the agent improved its allocation policy during training.

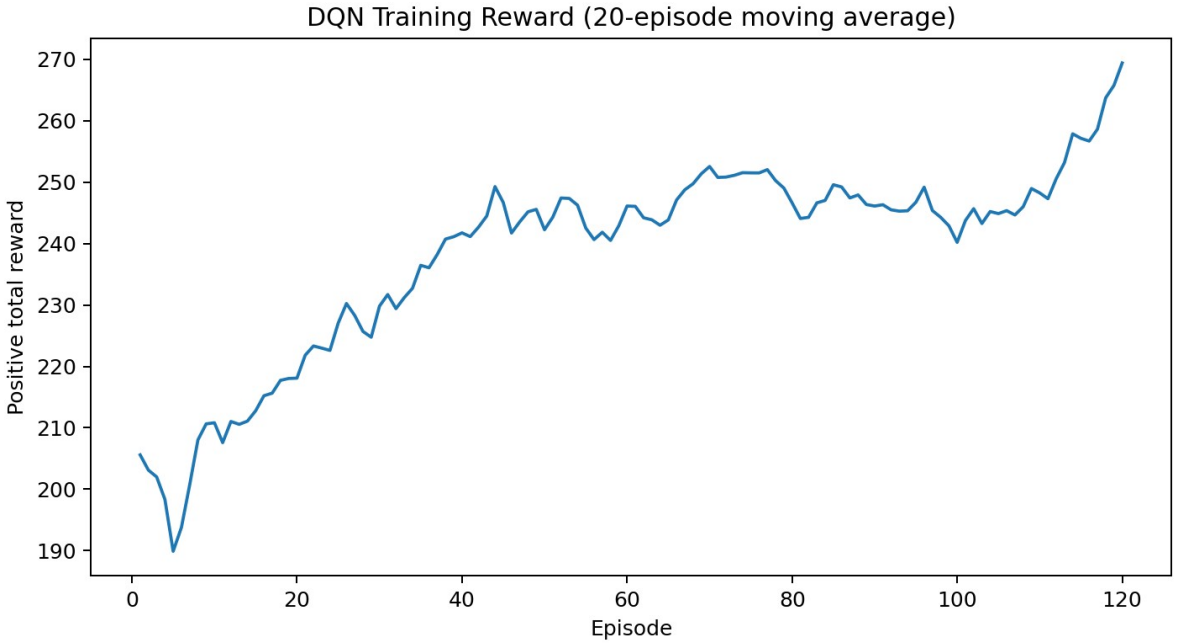


*Figure 1: DQN training reward moving average. The upward trend indicates learning progress.*

## 5. Non-RL Baselines

The deliverable requires comparison with at least one non-RL baseline. This project includes five non-RL baselines, which strengthens the evaluation because each baseline represents a different manual or heuristic allocation rule.

| Baseline | Rule / Purpose |
|---|---|
| First-Come-First-Served | Serves recipients in a fixed queue order. This is the main realistic manual baseline. |
| Greedy Expiry | Prioritizes food that is closest to expiry. |
| Nutrition-First | Prioritizes allocations with highest raw nutrition value. |
| Fairness-First | Prioritizes under-covered families to reduce inequality. |
| Random | Random action selection, used as a lower-bound reference. |

## 6. Experimental Results

The final results are averaged over repeated evaluation episodes. Higher reward, nutrition and mean coverage are better. Lower expired units, fairness gap and mismatches are better. The DQN agent achieved the highest overall positive reward, while remaining competitive on nutrition, fairness and dietary compatibility.

## 6. Experimental Results

| Method | Reward | Nutrition | Expired Units | Fairness Gap | Mean Coverage | Mismatches |
|---|---|---|---|---|---|---|
| DQN RL Agent | 288.207 | 1719.102 | 36.960 | 0.615 | 0.251 | 2.580 |
| First-Come-First-Served Baseline | 210.304 | 1690.399 | 36.960 | 0.895 | 0.278 | 14.500 |
| Greedy Expiry Baseline | 251.966 | 1728.003 | 36.960 | 0.582 | 0.230 | 6.420 |
| Nutrition-First Baseline | 198.079 | 1854.473 | 36.960 | 1.698 | 0.129 | 1.520 |
| Fairness-First Baseline | 286.862 | 1719.035 | 36.960 | 0.597 | 0.251 | 2.760 |
| Random Baseline | 218.751 | 1439.234 | 37.900 | 0.753 | 0.190 | 3.660 |

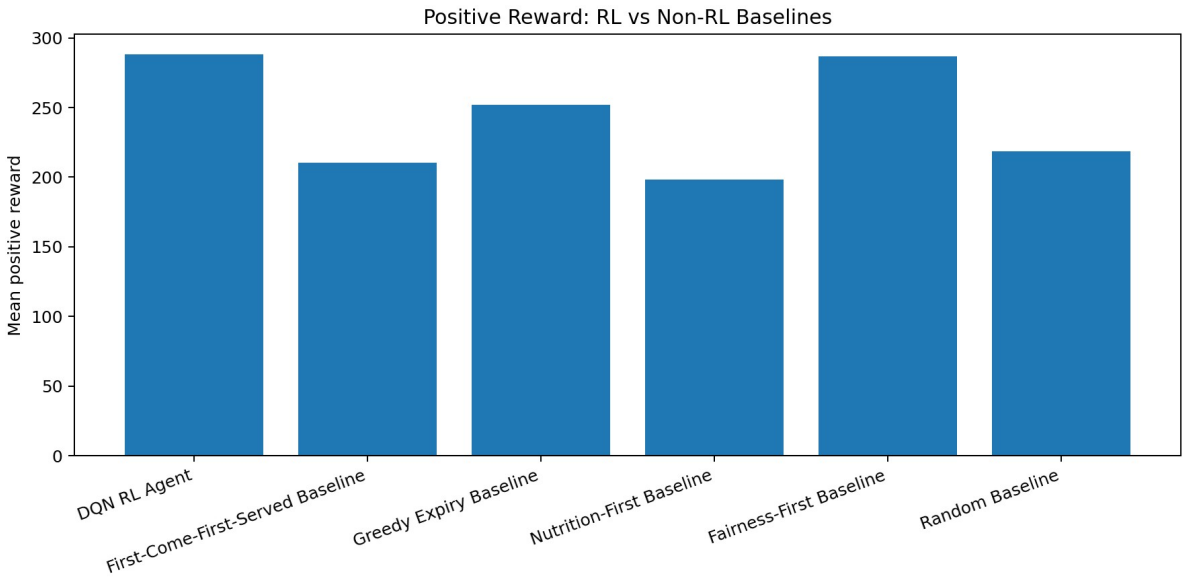


*Figure 2: Positive reward comparison. DQN achieved the highest mean reward.*

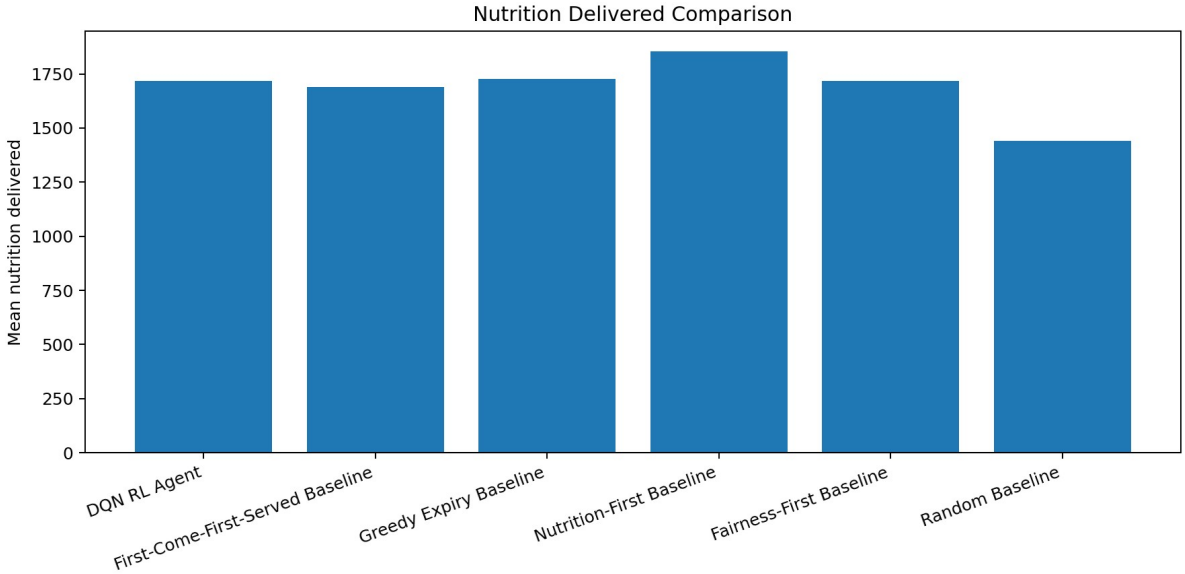


*Figure 3: Nutrition delivered comparison. Nutrition-First gives the highest raw nutrition, but does not balance fairness well.*

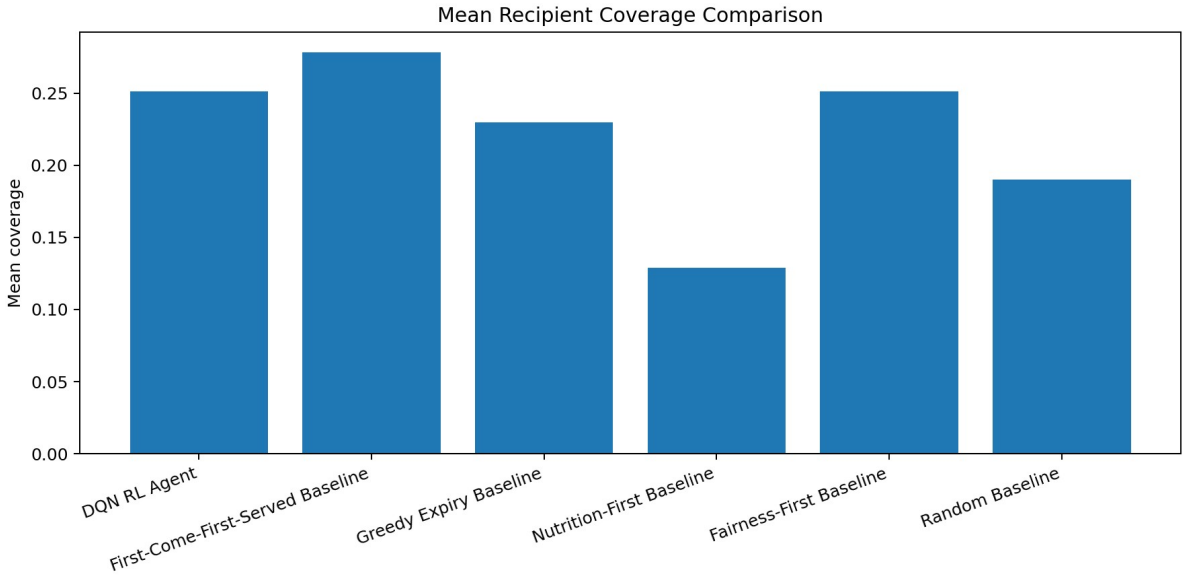


*Figure 4: Mean recipient coverage comparison.*

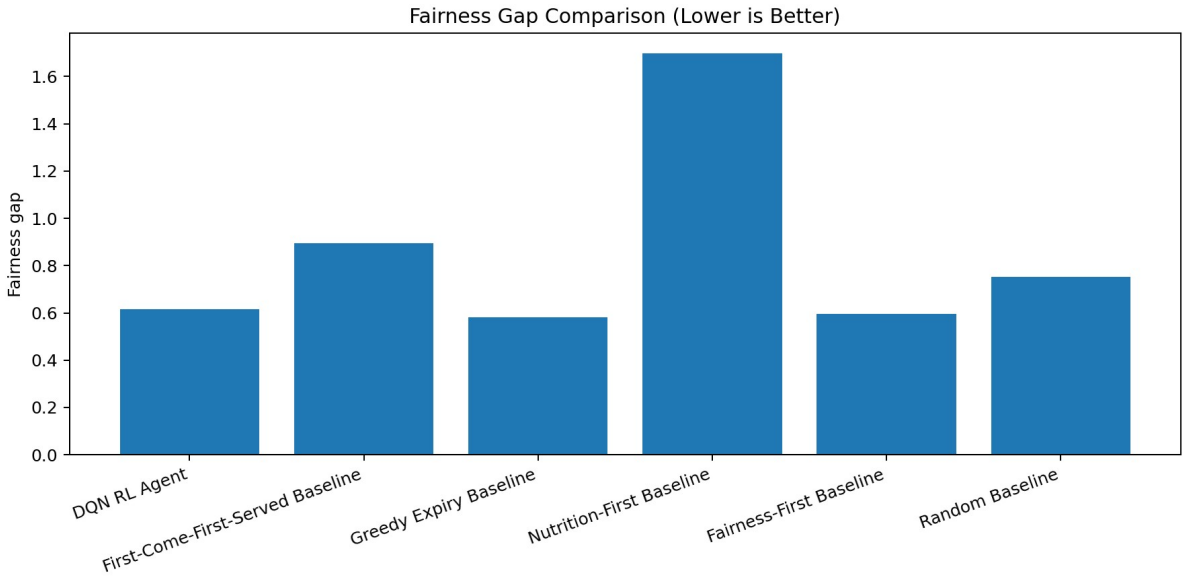


*Figure 5: Fairness gap comparison. Lower values indicate more equal distribution.*

## 7. Critical Analysis

The DQN agent reached a mean positive reward of 288.207, outperforming the strongest non-RL competitor, Fairness-First, which reached 286.862. This difference is small but meaningful because Fairness-First is specifically designed for this kind of problem, while DQN learned its policy from interaction with the environment.

The Nutrition-First baseline delivered the highest raw nutrition (1854.473), but it had the worst fairness gap (1.698) and the lowest mean coverage (0.129). This demonstrates why the objective cannot be nutrition alone. The DQN produces a balanced outcome: high nutrition, tied-low expired units, low mismatch count and competitive fairness.

First-Come-First-Served achieved strong mean coverage (0.278) but produced many mismatches (14.50), showing that queue-based manual allocation can be simple but may ignore dietary compatibility. The DQN reduced mismatches to 2.58, which is much safer for a dignity-aware food allocation system.

A limitation is that the environment is still a simulation. Real deployment would require real food-bank demand data, more detailed dietary categories, stochastic donation arrivals and route/storage constraints. However, for the course deliverable, the system demonstrates the full RL workflow: simulation, training, saved model, baselines, evaluation and browser demo.

## 8. How to Run the Prototype

- Install dependencies: pip install -r requirements.txt
- Train the DQN agent: python train.py
- Evaluate against baselines: python test.py
- Run the web app: python -m streamlit run app.py
- Use the included saved model at models/dqn_foodbank.pt to run testing/demo without retraining.

## 9. Individual Reflection Questions

| Question | Answer Summary |
|---|---|
| Specific individual contribution | Talha Noor supported baseline comparison planning, documentation review and demo validation. Sufiyan Nadeem worked on RL formulation, environment testing, DQN training/evaluation, reward tuning and dashboard verification. Ali Rana supported web interface testing, screenshots/result verification and live presentation preparation. |
| Most difficult technical challenge | The hardest challenge was designing a positive reward function that was still meaningful. This was solved by rewarding useful allocation behavior directly instead of using a penalty-heavy objective. |
| How AI tools were used | AI assistance was used to generate starter structure, debug PyTorch/Streamlit issues, tune the reward/action formulation, prepare report/presentation material and interpret evaluation outputs. The code and results were tested locally after generation. |
| New RL understanding | The project showed that reward design and action/state formulation are as important as the neural network. More training alone does not help if the environment is not learnable or the reward does not represent the actual goal. |

## 10. Conclusion

The project satisfies the prototype deliverable by providing a complete runnable RL system with source code, README instructions, web interface, training script, saved model, custom environment and multiple non-RL baselines. The final results show that the DQN agent achieved the best overall positive reward, proving that RL can learn a balanced food allocation strategy across nutrition, fairness, expiry and dietary constraints.

## References

Mnih, V. et al. Human-level control through deep reinforcement learning. Nature, 2015.

van Hasselt, H., Guez, A., and Silver, D. Deep Reinforcement Learning with Double Q-learning. AAAI, 2016.

Wang, Z. et al. Dueling Network Architectures for Deep Reinforcement Learning. ICML, 2016.

Sutton, R. S. and Barto, A. G. Reinforcement Learning: An Introduction, 2nd Edition. MIT Press, 2018.

Gymnasium Documentation. Farama Foundation.